University of Glasgow | School of Computing Science

Tuesday 15 May 2018
09.30 am – 11.00 am

(Duration: 1 hour 30 minutes)

EXAMINATION FOR

DEGREES OF MSci, MEng, BEng, BSc, MA and MA (Social Sciences)

Big Data (H)

Exam Rubric: Answer All Questions

This examination paper is worth a total of 75 marks

You must not leave the examination room within the first hour or the last half-hour of the examination.

INSTRUCTIONS TO INVIGILATORS

Please collect all exam question papers and return to School together with exam answer scripts

Question 1 — 20 Marks

A. Outline the steps that a read request goes through, starting when a client opens a file on HDFS (given a pathname) and ending when the requested data is in the client's memory buffer. Also explain what happens if there is a network error in the process.

[10 marks]

[Bookwork]
The client first opens the file. To do this, it contacts the NameNode (NN) with the requested pathname. The NN responds with the IDs and locations (DataNode (DN) IP addresses, ports) of the first few blocks of the file. The list of DNs per block is sorted based on their network distance (# network hops) to the client. [3 marks]
If there is a network error, the client will retry for a configurable amount of times or until it receives a response from the NN. [1 mark]
The client then contacts directly the DN storing the block of interest (say the 1st block of the file). The DN responds by sending back the checksum of the block and streams the data of the block. The latter means that the block is not sent in one chunk but rather in much smaller packets (typically 64Kbytes, when the block size is 64 or 128Mbytes). [2 marks]
When the block has been transferred, the client will verify its checksum. If the verification is successful, then the data becomes available to the application. [1 mark]
If the verification fails, the client will report it to the NN and will try with another DN in the list. [1 mark]
If the network connection fails while contacting/transferring data from the DN, then the client will try with the next DN in the list. It will also mark the former DN as faulty so as to avoid further failed reads in the session. [2 marks]

B. Explain how the in-memory filesystem image is restored to full working order when a HDFS NameNode starts up after a reboot.

[10 marks]

[Bookwork]
On restart, the NameNode (NN) will first read the checkpoint file and journal file from its local disk. [2 marks] The checkpoint file is a snapshot of the filesystem state at some point in the past. [1 mark] It will then replay those transactions from the journal file that are later than the checkpoint's timestamp. This will restore the directory structure and the mapping of blocks to files in the in-memory state of the NN. [2 marks]
The NN will then wait for all DataNodes (DNs) to register. [1 mark] While doing so, each DN sends back a list of the IDs of blocks it stores alongside their size. [2 marks] The NN will then store this information in its in-memory data structures, so that it can respond to clients with block locations per file. [2 marks]

Question 2 — 32 Marks

A. You are given a large unlabelled directed graph stored in a text file, in which each line contains two numbers:

<Source node ID> <Target node ID>

That is, each line represents a directed edge in the graph (assume that each edge appears only once in the file). You are requested to design a MapReduce job to output all unique node IDs and the number of edges pointing to each of them (in-links/in-edges); that is, the output should contain one line per node, formatted like so:

<Node ID> <Number of in-links>

Define what your job would do to accomplish the above; that is, define what is the format of the key-value pairs at both the input and output of your mappers and reducers, as well as the kind of processing that goes on at the mappers/reducers.

How would your design change if you were requested to output the above information but only for those nodes with the top-k largest number of in-edges, for relatively small values of k?

While designing your jobs try to minimize disk I/O and network bandwidth utilisation. You don't need to write code or pseudocode, but feel free to do so if it makes your explanation easier or clearer.

[18 marks total: 9 marks for the first job, 9 marks for the second (top-k) job]

[Synthesis/Problem solving]

We will use the same reasoning as that in the solution of the standard WordCount problem.

- Mappers:
  - Mapper input format: Each mapper will read the input one line at a time (i.e., each map() invocation will receive a single line of text). [1 mark]
  - Mapper processing and output format: For each line, it will extract the token for the target node ID, and will then emit a key-value pair where the key will be the target node ID and the value will be the number 1. [2 marks] Students can also opt to use a null value to reduce network overhead.
- Reducers:
  - Reducer input format: Each invocation of the reduce() function will receive a node ID as the key and a list of numbers (and/or null values) as its associated values. [1 mark]
  - Reducer processing and output format: It will then proceed to sum up the values and will emit a key-value pair where the key is the node ID and the value is the computed sum. Null values in this case will be deemed as having a value of 1. [2 marks]
- Combiners:
  - Combiners could be used, in essence doing the same as the reducers. [1 mark for proposing the use of combiners]
  - Combiner input format: each invocation of the combiner's reduce() function would receive a node ID as the key and a list of numbers (and/or null values) as the list of values. [1 mark]
  - Combiner processing and output format: it would then emit a key-value pair where the key would be the node ID and the value would be the sum of numbers in the set (null values considered as having a value of 1). [1 mark]

To compute the top-k result, the mappers and combiners would be the same but reducers on the other hand would need to be changed.

- Mappers/combiners:
  - Same as above. [1 marks]
- Reducers:
  - Reducer processing: Instead of directly emitting each key-value pair when computed, they would need to keep a sorted list in their memory (initialized by the setup() function), to which they would add every computed key-value, removing those that are past the k'th position. [3 marks]
  - Reducer output: They would then emit the elements of this list when their input would be depleted (i.e., as part of their cleanup() function). [2 marks]
- Post-processing: This would create as many k-sized output files as there would be reducers, which would require an extra step at the Driver to merge the results and return just the top-k. As the number of reducers is relatively small and configurable, and k is also relatively small, this task can easily be carried out by the Driver. [3 marks]

Students may opt to carry this operation out by first executing the above inedge counting job, then having a second job where mappers would maintain a local top-k list using a local sorted list (manipulated as above) which would be emitted by their cleanup() function, with the output sent to a single reducer that would simply select the top-k results (again, using a local sorted list manipulated as above). As this is clearly suboptimal (due to first executing a full WordCount-like pass over the input data, then a second pass over the intermediate results), this solution would be marked lower than the above. [7 marks, instead of the above 9 marks]
Other designs are acceptable but will be judged on the basis of their correctness and performance characteristics with regards to disk and network I/O.

B. Briefly explain what happens after a mapper emits a key-value pair and until that key-value pair ends up in a reducer function's input.

[14 marks]

[Bookwork]
The key-value pair is first stored in a memory buffer at the mapper. [1 mark]
When the buffer fills up, it is sorted by key (using the key class's comparator), partitioned (if there are multiple reducers) and written out to disk (spill) on the local filesystem. [2 marks]
The system may also choose to execute combiners on the output of the mapper to reduce the amount of data written to disk and subsequently transferred over the network. [1 mark]
When the map input has been depleted, all pre-sorted and pre-partitioned outputs are merged (sort-merge-join) into a large sorted and pre-partitioned file which constitutes the mapper's final output. [2 marks]
As soon as a mapper creates its final output, it notifies the system (JobTracker, through its TaskTracker heartbeat, or similarly the AppMaster through its NodeManager if using YARN). [2 marks]
The reducers waiting for the mapper's output (i.e., reducers assigned keys existing in the mapper's output) will be notified by the system (through their TaskTracker or NodeManager as above). [1 mark]
Each such reducer will then contact the mapper and will start transferring that partition of the mapper's output mapped to the former. [2 marks]
Incoming data will be merged (sort-merge join, possibly calling combiners again). If the data cannot fit in the reducer's memory, it will be written out to disk. [2 marks]
Once the merge phase finishes, its output is then fed to the reduce() function. [1 mark]

Question 3 — 23 Marks

A. All NoSQL stores covered in the course are designed for high write throughput. Explain what mechanisms are in place to accomplish this design goal while achieving durability and briefly discuss the related trade-offs for read and write operations.

[10 marks]

[Analysis/Synthesis]
High write throughput is achieved through a combination of buffering/batching and immutable writes, with write-ahead logging in place for durability. That is:

- Incoming data are first stored in an in-memory buffer. [1 mark]
- A record is appended to the write-ahead log per write operation. The WAL is persisted on disk. [1 mark]
- To avoid individual round-trips to the disk per write operation, WAL update operations are batched and writes are not acknowledged to the client unless the relevant WAL record has been persisted to disk. [2 marks]
- When the in-memory buffer fills up, it is written out to disk in a separate file per buffer spill. These files are then immutable, thus avoiding further delays due to disk seeks. [1 mark]

Trade-offs:

- The main shortcoming of this design is that it creates a large number of files on disk that need to be scanned and/or merged to produce the response to a subsequent read operation. This can lead to a higher read latency. [1 mark]
- Batching of WAL writes also leads to a higher write latency, but that is preferable to facing a lower write throughput which would result from writing out one item at a time. [1 mark]
- To avoid negatively impacting the system's read performance, these output files need to be periodically scanned/merged (compactions). [1 mark]
- This is usually done lazily (to avoid negative impacts on the system's read/write performance), but in the meanwhile the system's read performance degrades gradually. [1 mark]
- Caching of read blocks is then used to alleviate some of the performance problems. [1 mark]

B. Explain the on-disk storage format of a row with multiple columns/attributes and multiple versions per column/attribute in a NoSQL store such as HBase.

[5 marks]

[Bookwork]
HBase rows are unlike rows in typical SQL databases. In HBase, data are stored in key-value pairs, where the value part contains the column value (and its size in bytes), and the key is a composite of the rowkey, column name, column qualifier and timestamp (and their sizes in byte). [3 marks]
When stored on disk, these key-values are sorted by composite key. As such, all key-value pairs for the same rowkey are stored in a contiguous part of the file. [2 marks]

C. List the consistency levels supported by Cassandra for single-datacentre deployments and briefly explain their behaviour for read and write operations.

[8 marks]

[Bookwork]
Cassandra features the following consistency level (2 marks per level: one for each of read and write behaviour):

- ANY: Writes succeed when persisted on at least 1 node (including hinted handoff nodes); this level is not supported for reads.
- ONE: Writes succeed when persisted on at least 1 node (excluding hinted handoff nodes); reads return data received by first replica to respond.
- QUORUM: Writes succeed when persisted on at least N/2+1 replicas (N being the desired number of replicas of a data item); reads return the record with the most recent timestamp once at least N/2+1 replicas have responded.
- ALL: Writes succeed when persisted on all N replicas; reads return record with most recent timestamp when all replicas have responded.